

# Prophet Hacks Full Research and Execution Playbook

Research-grade forecasting system, monitoring plan, SAE-inspired calibration, frontier-model strategy, batch/offline experiments, agent coordination, and implementation roadmap

Prepared by

Rob Sneiderman - methodologist, statistician, data engineer, and Prophet Hacks competitor. This report is intended for Rob and any coding/research agents working on the repo.

Core thesis

Win by building a monitored probabilistic decision system, not a vibes-based chatbot: market prior + targeted evidence + frontier-model reasoning + borrowed-strength calibration + ensemble disagreement + conservative risk gate + complete traces.

## End-to-end operating loop



Version 6 - expanded from prior monitoring/SAE notes into a full plan with diagrams and implementation details.

# 0. How to use this document

This is the full playbook, not the short SAE memo. It consolidates the ideas from our discussion: frontier models, model averaging, batch jobs, sandboxed agents, GitHub convergence, RAG/messaging, monitoring, smoke tests, hyperparameter search, Kalshi microstructure lessons, Prophet Arena benchmark design, and small-area-estimation inspired calibration.

| Read first     | Why                                                                                                                                                                            |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sections 1-3   | Definitions and research context so the rest does not feel like disconnected agent jargon.                                                                                     |
| Sections 4-8   | The practical system: architecture, frontier-model policy, batch/API credits, sandbox/GitHub, and data collection.                                                             |
| Sections 9-13  | The statistical engine: SAE borrowing, Kalshi lessons, stacking, retrieval, prompts.                                                                                           |
| Sections 14-18 | Monitoring, smoke tests, offline eval, hyperparameter search, and RL-lite.                                                                                                     |
| Sections 19-25 | Risk, infrastructure, repo structure, roadmap, research artifact, and open questions.                                                                                          |
| Default stance | Start building now. Do not wait for live markets to exist. Pre-live work is monitoring, offline replay, calibration, tests, source gates, model adapters, and clean packaging. |

# 1. Definitions and vocabulary

---

The goal is to remove ambiguity. When agents work on the repo, they should use these terms consistently.

| Term                | Working definition for this project                                                                              |
|---------------------|------------------------------------------------------------------------------------------------------------------|
| Event               | The broad future question or container: CPI release, election, sports game, product launch, weather outcome.     |
| Market              | A specific Yes/No proposition under an event. The unit our bot forecasts and may trade.                          |
| Forecast            | A probability $p_{\text{yes}}$ that the market resolves Yes. Prose rationale is secondary; the number is scored. |
| $p_{\text{market}}$ | Market-implied probability, preferably derived from executable prices, not just a midpoint.                      |
| $p_{\text{model}}$  | Raw probability from GPT-5.5, Opus, or another forecaster before calibration.                                    |
| $p_{\text{final}}$  | The calibrated probability after market blending, ensemble pooling, SAE shrinkage, and guards.                   |
| YES edge            | $p_{\text{final}} - \text{yes\_ask}$ . This is executable edge for buying YES.                                   |
| NO edge             | $(1 - p_{\text{final}}) - \text{no\_ask}$ . This is executable edge for buying NO.                               |
| Brier score         | Squared error of probability forecasts. Lower is better. Random binary 0.5 forecast has expected 0.25.           |
| ECE                 | Expected calibration error: do events predicted at 70% happen about 70% of the time?                             |
| Stacking            | Combining forecasts from market, GPT, Opus, rules, and retrieval signals.                                        |
| SAE borrowing       | Partial pooling across related sparse groups: domain x horizon x price bucket.                                   |
| Trace               | JSONL record of every decision, including inputs, forecast, edge, action, skip reason, and cost.                 |

## 2. Problem frame: the system we are actually building

The wrong frame is: "ask an LLM what will happen." The right frame is: build a probabilistic decision system that can explain, monitor, and improve its forecasts.

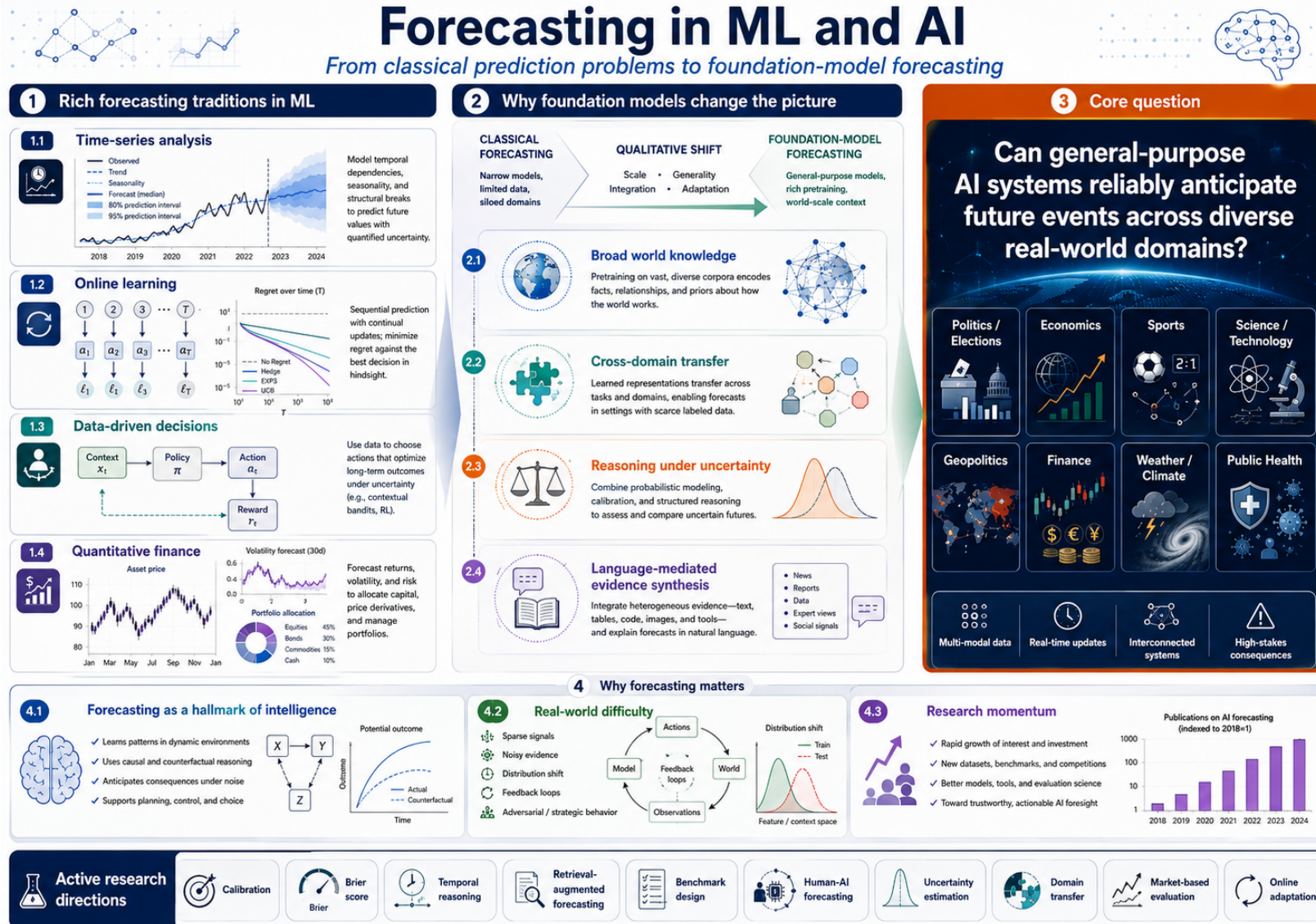
| What good looks like                                                                                                                                                                                                                             | What bad looks like                                                                                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Runs without babysitting; uses market prices as strong priors; uses frontier models where they matter; uses sources selectively with timestamps; produces calibrated probabilities; compares variants to market-only; logs every trade and skip. | Complex multi-agent debate with no diagnostics; raw LLM confidence deciding trade size; lots of data with no source-quality model; no price-bucket diagnostics; no clean smoke/package test; no market-only comparison; no trace of why a trade occurred. |
| <b>Research-grade framing</b>                                                                                                                                                                                                                    | The artifact should be publishable as a system design and evaluation study: market baselines, calibrated forecasts, source quality, ensemble disagreement, monitoring, and borrowed-strength calibration.                                                 |

### 3. Source map: what our strategy is grounded in

| Source                                 | What it contributes                                                                                                                                   | Agent consequence                                                                                        |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Prophet Arena / LLM-as-a-Prophet paper | Live forecasting pipeline; market baseline; Brier, ECE, market return; source-quality ablations; model conservatism; reasoning-to-probability issues. | Mirror the modular pipeline and log probability, market prior, source context, and rationale separately. |
| Kalshi paper                           | Favorite-longshot bias; maker/taker microstructure; cheap contracts underperform; market prices informative but biased.                               | Add longshot guard, price-bucket diagnostics, executable edge, and spread-aware thresholds.              |
| Baseline strategy                      | Simple ai-prophet-core loop; edge formula; conservative risk caps; no big multi-agent system first.                                                   | Keep first live bot robust and observable.                                                               |
| Budget policy                          | Model/API cost caps; strong models only for selected final synthesis; no open-ended loops; log costs.                                                 | Use frontier models deliberately; use batch/offline evals to spend credits rationally.                   |
| Operating pack                         | Robust, reproducible, easy-to-debug submission; pin packages; one command; dry tick before complexity.                                                | Build repo hygiene and packaging early.                                                                  |
| Research spine                         | Topics: scoring rules, calibration, online learning, prediction markets, retrieval credibility, observability.                                        | Use research as implementation scaffolding, not a reading distraction.                                   |
| Submission checklist                   | Env vars, lifecycle, risk, observability, packaging, post-event artifacts.                                                                            | Turn checklist into CI gates.                                                                            |

## 4. Visual landscape: Forecasting in ML and AI

Landscape infographic created earlier: forecasting traditions, foundation-model shift, core cross-domain question, why forecasting matters, and active research directions.





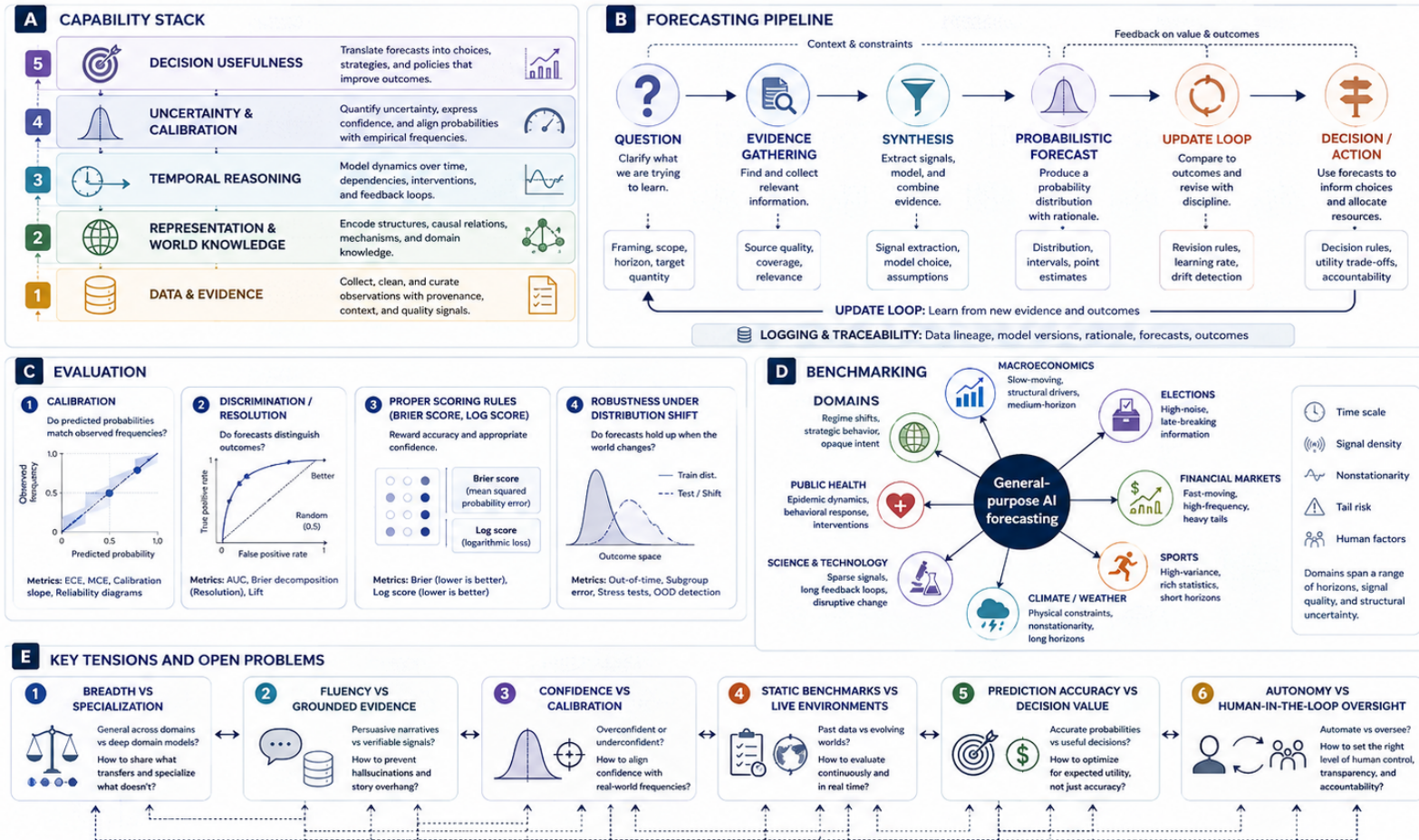
## 5. Visual landscape: AI forecasting research map

Research landscape infographic created earlier: capability stack, forecasting pipeline, evaluation, benchmarking domains, tensions, and open problems.

# AI Forecasting Research Landscape

Capabilities, evaluation, and system design for anticipating the future

Forecasting sits at the intersection of prediction, reasoning, uncertainty, and action.



The central challenge is not merely producing forecasts, but producing calibrated, updateable, decision-relevant probabilities across diverse real-world settings.

## 6. Workshop topics -> system modules

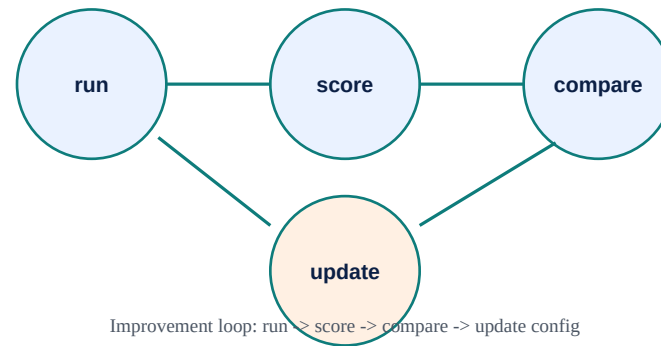
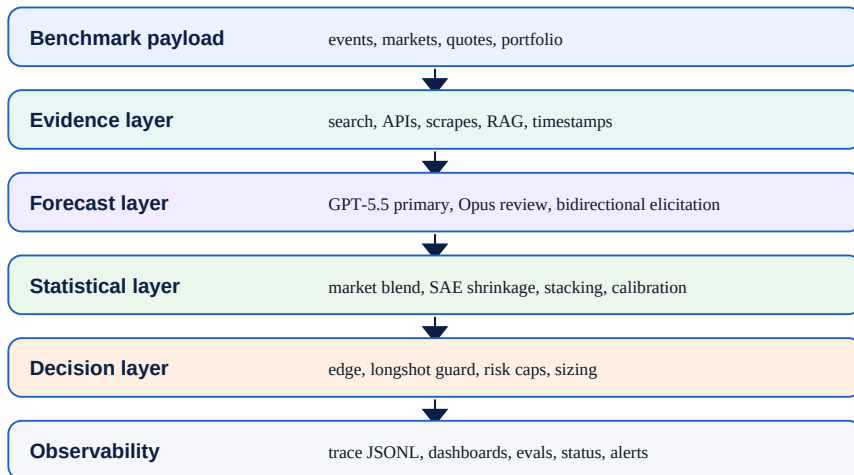
The workshop topics become concrete modules. This prevents the project from becoming abstract research without code.

| Workshop topic                                                            | System module                                                               | Concrete deliverable                                    |
|---------------------------------------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------|
| Architectures: agentic systems, LLM-as-a-Prophet, foundation/world models | Forecasting pipeline + frontier provider adapters + agent sandbox protocol  | `forecasting/`, `providers.py`, `agent_protocol.md`     |
| Evaluation: event generation, metrics, benchmarks                         | Offline replay + Brier/ECE/return + reliability diagrams                    | `tools/evaluate_offline.py`, `reports/eval/`            |
| Reasoning: probabilistic, calibration, causal/temporal inference          | Structured prompts + bidirectional elicitation + calibration layer          | `prompts.py`, `calibration.py`, `bidirectional.py`      |
| Retrieval: search, credibility, RAG                                       | Source gate, source-quality model, separate RAG stores                      | `source_gate.py`, `source_scoring.py`, `ask_project.py` |
| Foundations: scoring rules, online learning, decision theory              | Brier/ECE, Hedge expert pool, CRRA/Kelly-lite risk                          | `metrics.py`, `expert_pool.py`, `risk.py`               |
| Markets & Society: prediction markets, impacts                            | Kalshi/Polymarket connectors as optional evidence; price-bucket diagnostics | `connectors/`, `longshot_guard.py`                      |



## 7. System architecture

### Architecture: monitored probabilistic decision system



Core design principle: keep forecast generation, calibration, trading action, and monitoring separate. The model can reason; code should enforce probability hygiene, price math, risk limits, and logging.

| Layer             | Owns                                                     | Should not own                             |
|-------------------|----------------------------------------------------------|--------------------------------------------|
| Evidence layer    | queries, APIs/scrapes, timestamps, source scoring        | final probability or trade size            |
| Forecast layer    | structured p_yes, uncertainty notes, base-rate reasoning | risk caps or execution                     |
| Statistical layer | market blend, stacking, shrinkage, calibration           | source crawling or hidden manual overrides |
| Decision layer    | edge, threshold, sizing, skip/trade                      | forecast rationale                         |
| Observability     | trace schema, dashboards, CI, status docs                | strategy logic                             |

# 8. Frontier-model policy: no, we are not downgrading the core forecaster

The default final synthesis should be frontier-first. The reason to mention cheap/non-frontier models is not because they are preferred for final forecasts; it is because they can reduce waste on obvious skips, source classification, and offline ablations.

| Use frontier models for                                                                                                                                                                                                                   | Use cheaper models/rules for                                                                                                                                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Primary final forecast synthesis; near-threshold markets; conflicting evidence; high-value/high-liquidity markets; hard domains such as politics, macro, geopolitics, public health, science/tech; LLM-as-judge audits on sampled traces. | Domain routing; obvious skip decisions; source type labels; format validation; bulk offline tests before frontier spend; ablation controls and sanity baselines.                                                          |
| Default route                                                                                                                                                                                                                             | GPT-5.5 primary forecast; Claude Opus cross-check only when expected value, disagreement, source conflict, or uncertainty justifies it. Other models are routers, controls, or ensemble members after evidence they help. |

OpenAI docs currently expose "Latest: GPT-5.5" in the API navigation and include Batch/cost optimization and agent sandbox/orchestration docs; Anthropic docs list Claude model families. Exact model names should remain configurable because provider catalogs change.

# 9. Batch jobs, API credits, and experiment spend

Batch jobs are for offline learning, not live ticks. Live forecasting needs low latency and current context; batch mode is perfect for replay, ablation, calibration-data generation, and agent audits.

| Use case                                  | Synchronous live API | Batch/offline API |
|-------------------------------------------|----------------------|-------------------|
| Live tick forecast                        | YES                  | NO                |
| Prompt ablation over 500 historical tasks | NO                   | YES               |
| Bidirectional p_yes/p_no test             | selected only        | YES               |
| Source-quality label generation           | maybe                | YES               |
| LLM-as-judge audit                        | sampled only         | YES               |
| Embedding/index construction              | NO                   | YES               |
| Hyperparameter search forecasts           | NO                   | YES               |
| Final trade decision                      | YES                  | NO                |

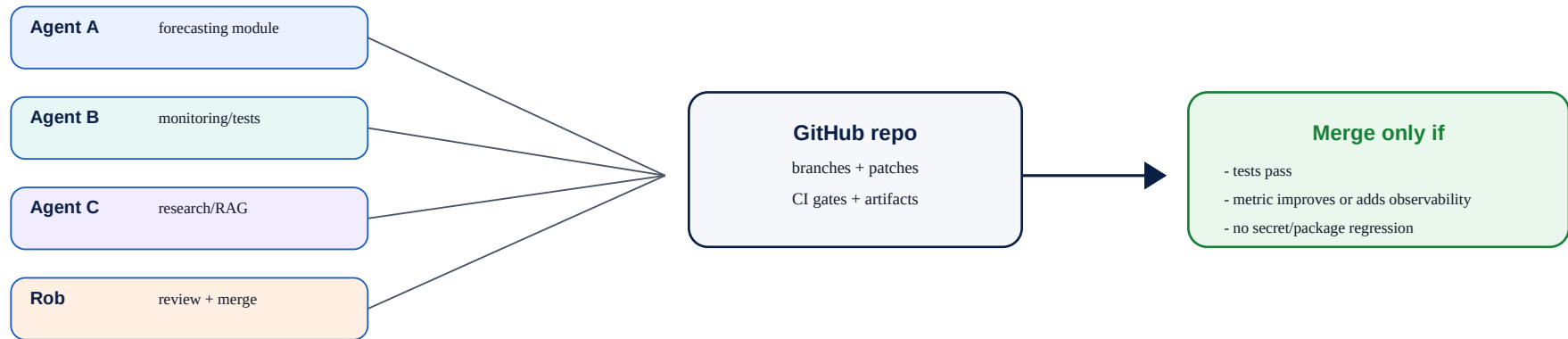
Credit policy

Spend credits to answer specific experiments: does retrieval help, does bidirectional elicitation improve ECE, does Opus review improve near-threshold accuracy, does SAE shrinkage improve bucket calibration? Do not spend credits just to make the system feel more agentic.

```
# Example experiment cap
max_batch_forecasts: 500
max_frontier_calls_per_tick: 6
max_opus_escalations_per_tick: 2
max_cost_per_offline_eval_usd: 75
max_cost_per_live_tick_usd: 3
```

## 10. Sandbox and GitHub convergence

### Agent sandbox and GitHub convergence model



The right sandbox is not a separate empire of agents. It is a disciplined workflow where agents propose patches, run commands, attach artifacts, and converge through GitHub. GitHub is the source of truth.

| Artifact          | Purpose                                                                                        |
|-------------------|------------------------------------------------------------------------------------------------|
| Branch/patch      | Isolate one change from one agent or one experiment.                                           |
| Commit message    | State what changed and why.                                                                    |
| CI artifact       | Attach trace summary, evaluation report, and charts.                                           |
| reports/status.md | Human-readable current state.                                                                  |
| Agent context MD  | Compact instruction file for future agents.                                                    |
| Merge rule        | Pass verify; either improve metric or add needed observability; no secret/package regressions. |

## 11. RAG, messaging, and control plane

A message system is useful if it creates operational clarity. The first version should be one-way summaries and local project Q&A;, not a fragile interactive bot.

| Channel               | Use                                                            | Priority    |
|-----------------------|----------------------------------------------------------------|-------------|
| GitHub Actions        | verify, offline eval, package checks, artifact upload          | High        |
| Slack webhook         | one-way status: pass/fail, offline eval summary, cost warnings | Medium      |
| Local CLI RAG         | ask questions about repo, papers, status, run logs             | High        |
| Interactive Slack bot | later; only after local CLI RAG is useful                      | Low         |
| Discord monitor       | only if it captures announcements or high-signal contest notes | Situational |

### RAG separation

- Ops RAG: README, runbooks, CI summaries, status.md, errors.
- Research RAG: Prophet paper, Kalshi paper, playbooks, notes.
- Forecast Evidence RAG: timestamped sources tied to event\_id/market\_id only.
- Never mix stale research memory with live evidence without timestamps.

## 12. Data collection: scraping, APIs, and usefulness

This section is intentionally practical. The decision is not "scrape or never scrape." The decision is whether a connector produces forecasting signal, operational awareness, or better offline evaluation. If it does not, it is noise.

| Source                     | Potential value                                               | Failure mode                                                  | Default                                |
|----------------------------|---------------------------------------------------------------|---------------------------------------------------------------|----------------------------------------|
| Prophet Arena payload      | source of truth for benchmark events, quotes, outcomes        | none if API used correctly                                    | Always primary                         |
| Discord/organizer messages | announcements, rule changes, live issues                      | chat noise, stale rumors, fragile scraping                    | Manual notes first; bot only if useful |
| Kalshi API/data            | external priors, microstructure, pastcast data, price buckets | not same as Prophet evaluation; overfitting to outside market | Optional research connector            |
| Polymarket API/data        | cross-market comparison, liquidity/consensus signal           | domain mismatch, wallet/trading distractions                  | Optional read-only connector           |
| Targeted web scrapes       | official calendars, scores, weather, financial releases       | source fragility and recency errors                           | Use for high-signal domains            |
| Broad crawling             | coverage                                                      | cost, latency, garbage source overload                        | Avoid for live; possible offline only  |

|                       |                                                                                                                                                                                                                            |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Connector rule</b> | Every external record needs source_id, URL/API endpoint, retrieved_at, event_id/market_id, source_type, quality score, staleness score, and whether it changed p_final. If it does not enter the trace, it does not exist. |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

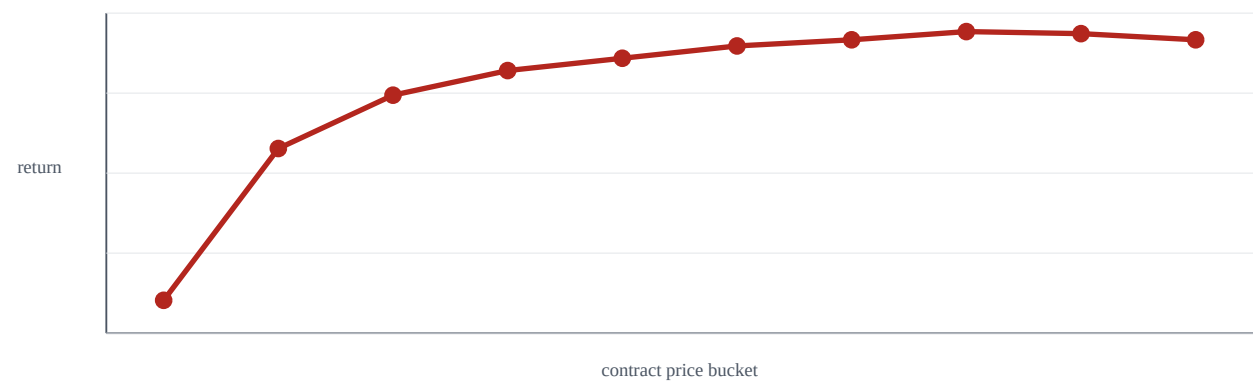


## 13. Prophet Arena lessons to implement

| Paper finding                                                                                  | Implementation consequence                                                     |
|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Forecasting is modular: event extraction -> context -> probabilistic prediction -> evaluation. | Keep repo modular: router, evidence, forecaster, calibrator, risk, monitoring. |
| Market baseline is a serious benchmark.                                                        | Always log and compare against p_market.                                       |
| Brier, ECE, and market return differ.                                                          | Dashboard all three; do not optimize only PnL.                                 |
| Market data + sources often best, but source effects are heterogeneous.                        | Use source gate and source quality, not blanket retrieval.                     |
| Models can be conservative relative to markets.                                                | Do not automatically shrink strong market favorites to 0.50.                   |
| Markets update faster near resolution.                                                         | Increase market weight near close.                                             |
| Reasoning-to-probability alignment is a bottleneck.                                            | Prompt for structured reasoning, but code computes final blend and risk.       |
| Bidirectional elicitation improved calibration in ablations.                                   | Implement p_yes and p_no option for selected markets.                          |

# 14. Kalshi lessons and market microstructure

## Kalshi-inspired diagnostic: price bucket vs expected return



**Longshot guard**  
If  $p_{\text{market}} < 0.10$ , require primary evidence + wider edge.

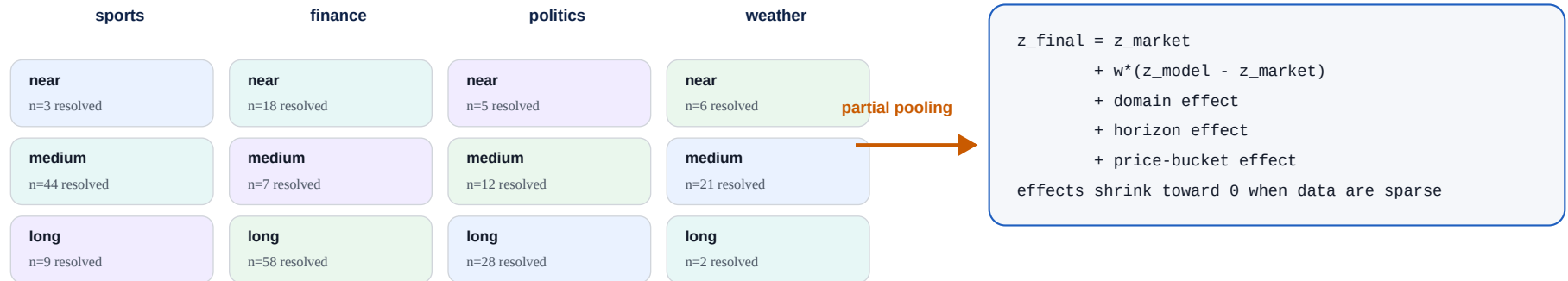
**Bucket diagnostics**  
Report Brier/return by price bin.

The Kalshi paper is useful because it shows prediction-market prices are informative but biased. The most operationally important result is the favorite-longshot pattern: cheap contracts underperform while high-price contracts can have small positive returns. This is exactly the kind of bias LLMs may amplify if they chase vivid low-probability narratives.

| Lesson                                                                      | Rule for bot                                                                        |
|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Cheap longshots underperform                                                | If $p_{\text{market}} < 0.10$ or $0.15$ , require stronger evidence and wider edge. |
| Market accuracy improves near close                                         | Near resolution, increase market-prior weight and reduce model deviations.          |
| Maker/taker spread matters                                                  | Compute edge using executable asks, not midpoint fantasies.                         |
| Heterogeneous beliefs and overweighting small probabilities can create bias | Treat model disagreement as uncertainty, not signal.                                |
| Higher-price favorites may be underpriced                                   | Do not reflexively pull strong favorites back to $0.50$ .                           |

## 15. SAE-inspired borrowed strength

### SAE-inspired borrowed strength: sparse cells shrink toward shared structure



Rob's small area estimation background is not just a metaphor. The forecasting problem has sparse cells: domain x horizon x price bucket x source-quality bucket. Direct estimates in each cell are noisy; global estimates ignore important structure. SAE says: partially pool.

```
# Sparse areas for calibration
area = (domain, horizon_bucket, price_bucket)

# empirical-Bayes shrinkage idea
area_correction = reliability(area) * direct_area_estimate

# reliability grows with resolved examples and signal quality
reliability = n_area / (n_area + shrink_k)
```

$$+ (1 - \text{reliability}(\text{area})) * \text{global\_estimate}$$

# 16. SAE implementation sketch

| Component          | SAE analogue                          | Implementation                                                               |
|--------------------|---------------------------------------|------------------------------------------------------------------------------|
| Direct estimate    | Noisy small-area estimate             | Observed calibration error or Brier skill in a domain/horizon/price bucket.  |
| Global model       | Synthetic/regression estimator        | Market blend + frontier forecast + source quality features.                  |
| Area random effect | Local deviation from global structure | Domain/horizon/price-bucket correction.                                      |
| Shrinkage factor   | Reliability of direct estimate        | $n\_area / (n\_area + k)$ , with stronger shrinkage for sparse cells.        |
| Borrowed strength  | Pooling across related areas          | Similar domains/horizons share strength rather than independent overfitting. |

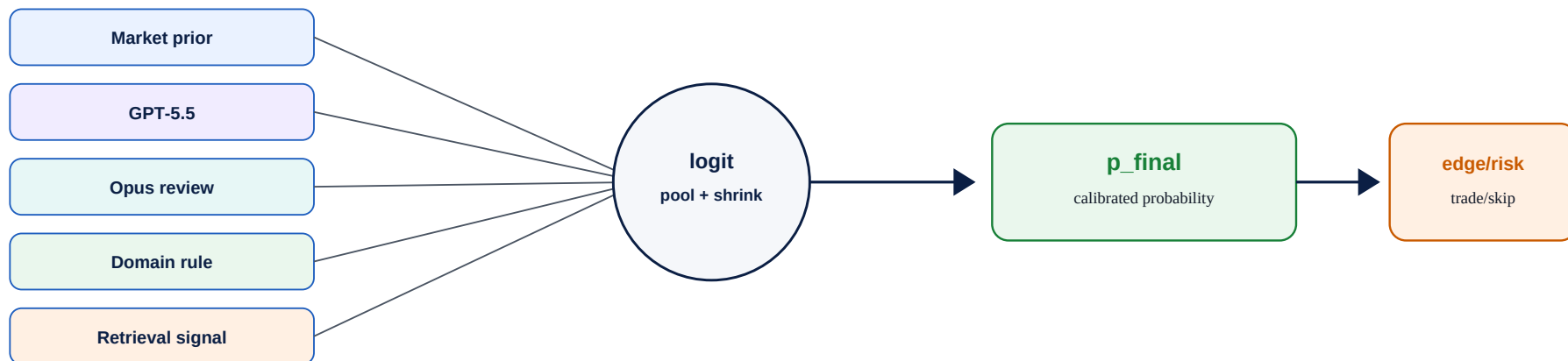
```
# first shippable version: not full Bayes, just disciplined shrinkage
z_final = logit(p_market)
z_final += w_model * (logit(p_model) - logit(p_market))
z_final += alpha_domain[domain] * reliability_domain
z_final += alpha_horizon[horizon_bucket] * reliability_horizon
z_final += alpha_price[price_bucket] * reliability_price
z_final -= beta_disagree * model_disagreement
p_final = sigmoid(z_final)
```

**Important**

SAE-inspired calibration should initially be offline/frozen. Live updating can be enabled only after confirming whether live adaptation is allowed and after offline tests show it helps.

## 17. Stacking, averaging, and disagreement

### Stacking and calibration: models are noisy estimators, not oracles



A single model probability is a noisy measurement. The final forecast should pool estimates in logit space and use disagreement as an uncertainty penalty.

```
p_final = sigmoid(
    w_market * logit(p_market)
    + w_gpt * logit(p_gpt)
    + w_opus * logit(p_opus)
    + w_rules * logit(p_domain_rule)
)

if stdev([p_gpt, p_opus, p_market]) > disagreement_threshold:
    shrink toward p_market or skip
```

| Stacker                         | When to use                                                   |
|---------------------------------|---------------------------------------------------------------|
| Fixed logit weights             | Default before enough resolved offline data.                  |
| Regularized logistic regression | After enough pastcast predictions with outcomes.              |
| Hedge/expert weights            | RL-lite after outcomes resolve; disabled live unless allowed. |
| Median/logit median             | Robust fallback when one model is erratic.                    |

# 18. Retrieval and source-quality scoring

Retrieval is not "more context." Retrieval is a measurement process. Bad sources can worsen forecasts. The Prophet paper's Bitcoin case study shows source quality can clarify or confound.

| Feature       | Scoring idea                                                                           |
|---------------|----------------------------------------------------------------------------------------|
| source_type   | official/primary > reputable current news > expert analysis > blog/social/forecast bot |
| recency       | source timestamp must be compatible with forecast_time and resolution horizon          |
| specificity   | source must address the exact market, not just the general topic                       |
| conflict      | conflicting sources increase uncertainty, not automatically a stronger signal          |
| actionability | does the source move base rate, mechanism, current state, or resolution criterion?     |

```
source_record = {
  "url": "...",
  "retrieved_at": "...",
  "event_id": "...",
  "market_id": "...",
  "source_type": "official|primary|news|analysis|social|bot",
  "credibility": 0.0,
  "staleness": 0.0,
  "supports_yes": false,
  "supports_no": false,
  "summary": "..."
```

# 19. Forecast prompt and structured output

The model should reason. Code should calibrate. Do not ask the model to choose trade size.

```
Return strict JSON:
{
  "base_rate": 0.0,
  "base_rate_note": "short",
  "evidence_for_yes": [...],
  "evidence_for_no": [...],
  "weak_or_stale_evidence": [...],
  "key_uncertainties": [...],
  "source_quality": 0.0,
  "raw_p_yes_before_market_blend": 0.0,
  "confidence": "low|medium|high",
  "reasoning_to_probability": "why this number follows",
  "trade_caution": "short"
}
```

| Field                         | Why it matters                                                              |
|-------------------------------|-----------------------------------------------------------------------------|
| base_rate                     | Prevents story-driven forecasts.                                            |
| evidence_for_yes/no           | Makes the rationale auditable.                                              |
| weak_or_stale_evidence        | Prevents retrieved garbage from silently influencing probability.           |
| source_quality                | Determines how much model signal can override market prior.                 |
| raw_p_yes_before_market_blend | Separates model belief from final calibrated p_final.                       |
| reasoning_to_probability      | Targets Prophet paper bottleneck: mapping reasoning to numeric probability. |

## 20. Toy example: from market prior to final action

Toy market: "Will Team A win tonight?" YES ask = 0.58, YES bid = 0.55. Market midpoint is 0.565; executable YES price is 0.58. GPT gives 0.67. Opus gives 0.62. Source quality is medium/high.

```
p_market = 0.565
p_gpt = 0.67
p_opus = 0.62
p_model = logit_median([0.67, 0.62]) ~= 0.645
p_final = blend_market_model(p_market, p_model, source_quality=0.70)
          ~= 0.61
yes_edge = 0.61 - 0.58 = 0.03
Decision: SKIP because edge < 0.08 threshold.
```

### Lesson

A good forecast does not automatically mean a trade. The forecast can be directionally useful but not executable after spread, uncertainty, and threshold.



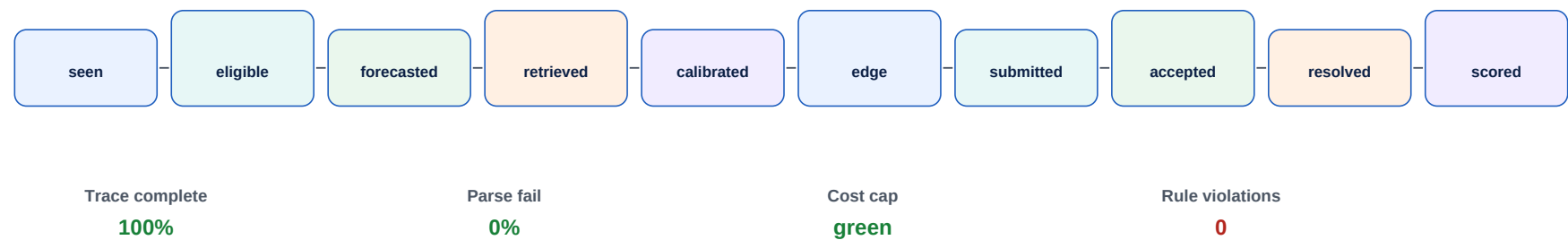
# 21. More realistic example: CPI-style market

Market: "Will April CPI inflation be above 0.3%?" This resembles the Kalshi paper example. The market has multiple numerical outcomes with spreads and related bins.

| Step              | Decision                                                                                                                                 |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Domain router     | Economics / scheduled data release / numerical threshold / near-term.                                                                    |
| Market prior      | Use YES/NO executable prices. Related bins imply monotonic constraints.                                                                  |
| Evidence          | Prefer BLS calendar, nowcasts, economist consensus, Fed-relevant releases; avoid generic macro commentary.                               |
| Consistency       | If $P(\text{CPI} > 0.4) > P(\text{CPI} > 0.3)$ , repair or reject forecast.                                                              |
| SAE borrowing     | Borrow from economics/scheduled-release/horizon bucket, not from sports or entertainment.                                                |
| Risk gate         | Trade only if threshold edge survives spread and monotonic consistency.                                                                  |
| Numerical markets | Use logical-chain constraints. Nested thresholds must be monotone. This is a paper-derived consistency test and a practical bug catcher. |

## 22. Monitoring: how Rob knows if we are on the right path

### Monitoring funnel: how Rob knows whether the system is working



Do not evaluate progress by how impressive the agent sounds. Evaluate the funnel. A good system tells us where it failed: filter, retrieval, forecast, calibration, edge, risk, execution, or scoring.

| Metric             | Green condition                                                                         |
|--------------------|-----------------------------------------------------------------------------------------|
| Trace completeness | 100% of decisions have market_id, p_market, p_model/p_final, edge, action, skip reason. |
| JSON parse failure | 0%, or fail clearly and skip.                                                           |
| Cost               | Within configured cap per tick and per offline run.                                     |
| Brier vs market    | At least near market-only first; then beat it on selected domains/horizons.             |
| ECE                | Improving or stable by bucket.                                                          |
| Trade rejects      | 0 or explained.                                                                         |
| Longshot bucket    | No uncontrolled low-price exposure.                                                     |

## 23. Smoke tests and test ladder

| Level | Test                          | Command idea                                                          |
|-------|-------------------------------|-----------------------------------------------------------------------|
| 0     | imports/config/risk constants | python -c "import agent, risk, forecaster"                            |
| 1     | dry-run startup               | python agent.py --slug smoke --dry-run                                |
| 2     | toy replay                    | python examples/monitoring_demo.py --events examples/toy_events.jsonl |
| 3     | offline pastcast              | python tools/evaluate_offline.py --dataset data/pastcast.jsonl        |
| 4     | package dry run               | python tools/package_submission.py --dry-run                          |
| 5     | live test slug when available | python agent.py --slug rob-live-smoke                                 |
| 6     | final clean machine test      | unzip submission && run one command                                   |

```
make verify
make offline-eval
make package-dry-run
make monitor TRACE=trace/latest.jsonl
```

# 24. Offline replay and pastcasting

Before live markets, success means offline replay and monitoring. The offline dataset should replay resolved tasks as if forecast\_time occurred before resolution\_time. Avoid future leakage.

| Variant            | Question answered                                 |
|--------------------|---------------------------------------------------|
| market_only        | How strong is the benchmark prior?                |
| gpt55_no_retrieval | Can frontier model add signal without sources?    |
| gpt55_with_sources | Does retrieval help or confound?                  |
| bidirectional      | Does p_yes/p_no improve calibration?              |
| opus_review        | Is cross-check worth cost near threshold?         |
| calibrated_stack   | Does market blend + shrinkage beat raw forecasts? |
| sae_shrinkage      | Does borrowed strength improve sparse buckets?    |

```
# Offline split discipline
train: older resolved events
validation: newer resolved events
holdout: newest pre-live resolved events
live: untouched official evaluation

# Never random-split temporal event data for final claims.
```

# 25. Hyperparameter search

Search is useful if constrained. Do not let Optuna or random search optimize only PnL on a tiny holdout; that will overfit. Optimize Brier skill, calibration, cost, and risk together.

| Family         | Parameters to search                                           |
|----------------|----------------------------------------------------------------|
| market blend   | model_weight_min/max, shrink_tau, horizon_market_weight        |
| risk           | base_edge_threshold, disagreement_skip_threshold, max_notional |
| retrieval      | source_cap, min_source_quality, stale_penalty                  |
| longshot guard | p_market_threshold, max_lift_without_primary_source            |
| escalation     | near_edge_margin, model_market_disagreement_threshold          |
| SAE shrinkage  | shrink_k by bucket, min_area_n, area grouping level            |

```
objective = (  
  - brier_skill_vs_market  
  + 0.5 * ece  
  + 0.1 * cost_per_100_forecasts  
  + 0.5 * max_drawdown_penalty  
  + parse_failure_penalty  
)
```

# 26. RL-lite and online learning

Full deep RL is not the right first move. RL-lite is useful as online expert weighting and routing: which model, whether to retrieve, whether to escalate, and how strongly to shrink.

```
# Hedge-style expert update after outcomes resolve
weight_i <- weight_i * exp(-eta * brier_loss_i)
normalize(weights)

experts = [market_only, gpt55, gpt55_sources, opus_review, calibrated_stack]

# Default: train/freeze offline.
# Live updates: enable only if contest rules and design allow it.
```

| Learns                       | Does not learn initially     |
|------------------------------|------------------------------|
| model weights                | direct neural trading policy |
| retrieval/escalation routing | reckless live exploration    |
| domain/horizon calibration   | unbounded strategy changes   |
| threshold adjustments        | opaque autonomous behavior   |

# 27. Risk and trading policy

Forecasting quality and trading quality are not the same. A forecast can be good but not tradable after spread. Trading code should be deterministic and stricter than server limits.

| Rule                      | Initial value / logic                                                                                           |
|---------------------------|-----------------------------------------------------------------------------------------------------------------|
| markets analyzed per tick | 5                                                                                                               |
| trades per tick           | 3                                                                                                               |
| new notional              | 100 USD max initially                                                                                           |
| edge threshold            | 0.08 base; higher if disagreement/spread/staleness is high                                                      |
| YES edge                  | $p_{final} - yes\_ask$                                                                                          |
| NO edge                   | $(1 - p_{final}) - no\_ask$                                                                                     |
| conflict guard            | never hold both YES and NO on same market                                                                       |
| longshot guard            | stronger evidence required when $p_{market} < 0.10$ or $0.15$                                                   |
| stale/unclear markets     | skip with explicit reason                                                                                       |
| Code owns action          | The model gives probability and uncertainty. Code computes edge, checks rules, sets size, and logs action/skip. |

# 28. Infrastructure decision tree

Default infrastructure should stay simple: MacBook + GitHub + local logs + API calls. Add hosting only if evaluation or continuous trading requires it.

| Question                 | Decision                                                                                  |
|--------------------------|-------------------------------------------------------------------------------------------|
| Need GPU?                | No for API-based forecasting. Only for local vLLM experiments or large batch inference.   |
| Need second MacBook/SSH? | Not by default. Useful only if you want a stable always-on runner or parallel local jobs. |
| Need webserver?          | Only if hosted endpoint submission is required or a dashboard becomes useful.             |
| Need MCP?                | Useful for development/control plane; do not make submitted agent depend on it.           |
| Need GitHub Actions?     | Yes, for verify/offline-eval/package artifacts.                                           |
| Need Slack?              | Optional one-way alerts. Interactive bot later.                                           |



## 29. Proposed repo structure

---

```
agent.py
config.yaml
README.md
SUBMISSION_NOTES.md
COMPLIANCE.md
reports/status.md

forecasting/
  providers.py
  prompts.py
  domain_router.py
  source_gate.py
  source_scoring.py
  forecaster.py
  bidirectional.py
  ensemble.py
  market_blend.py
  calibration.py
  sae_shrinkage.py
  longshot_guard.py

trading/
  edge.py
  sizing.py
  risk.py

evaluation/
  brier.py
  ece.py
  returns.py
  reliability.py
  compare_variants.py

tools/
  monitor_run.py
  evaluate_offline.py
  hyperparam_search.py
  check_no_secrets.py
  check_compliance.py
  ask_project.py

tests/
  test_edge_math.py
  test_market_blend.py
  test_longshot_guard.py
  test_metrics.py
  test_trace_schema.py
.github/workflows/
  verify.yml
  offline-eval.yml
```

## 30. Reports and dashboards

---

The status report is Rob's control panel. It is the antidote to feeling lost.

```
# reports/status.md
Owner: Rob Sneiderman
Default variant:
Last green command:
Current blocker:
Best offline Brier:
Market-only Brier:
ECE:
Trace completeness:
Parse failure rate:
Model cost per 100 forecasts:
Known red risks:
Next action:

# Every offline run should produce:
- monitor_summary.md
- brier_by_variant.png
- reliability_diagram.png
- decision_funnel.png
- price_bucket_diagnostics.png
- trace_sample.jsonl
```

# 31. Instructions for collaborating agents

| Agent behavior     | Required output                                                                                       |
|--------------------|-------------------------------------------------------------------------------------------------------|
| Implement a module | patch + tests + run command + known failure modes                                                     |
| Research a paper   | one-page implication memo + code idea + risk/uncertainty                                              |
| Run eval           | variant table + trace artifact + whether result changes default strategy                              |
| Change prompt      | before/after Brier/ECE/cost + parse failure rate                                                      |
| Add connector      | signal value + staleness model + trace fields + kill switch                                           |
| Propose merge      | why it improves score/observability + exact commands passed                                           |
| No invisible work  | Any agent output that does not include commands, artifacts, or a measurable delta is not merge-ready. |

## 32. Implementation roadmap

| Phase                | Goal                           | Deliverables                                    |
|----------------------|--------------------------------|-------------------------------------------------|
| 0. Repo hygiene      | Known green baseline           | verify workflow, no-secret check, status.md     |
| 1. Monitoring first  | Every run explainable          | trace schema, monitor_run, decision funnel      |
| 2. Offline replay    | Pre-live evaluation            | toy/pastcast dataset, Brier/ECE/return          |
| 3. Frontier forecast | Structured p_yes               | GPT-5.5 adapter, prompt schema, parser tests    |
| 4. Calibration       | Market blend and shrinkage     | market_blend, longshot_guard, reliability plots |
| 5. Ensemble          | Opus review and stacking       | ensemble.py, disagreement shrinkage             |
| 6. Source gate       | Selective retrieval/API/scrape | source scoring, source logs, RAG separation     |
| 7. SAE layer         | Borrowed strength              | sae_shrinkage, bucket diagnostics               |
| 8. Live hardening    | Survive benchmark              | resume, retry, package, clean run command       |

### 33. Research-grade artifact outline

Assume we win or at least produce a serious artifact. Capture everything so the result can become a report, blog, TheoremPath module, or paper.

| Section       | Content                                                                                            |
|---------------|----------------------------------------------------------------------------------------------------|
| Abstract      | Open-domain AI forecasting with market priors, frontier models, and borrowed-strength calibration. |
| Methods       | Pipeline, model routing, source gate, stacking, SAE shrinkage, risk gate.                          |
| Data          | Prophet Arena events, offline pastcast tasks, source records, price buckets.                       |
| Metrics       | Brier, ECE, market return, Sharpe, bucket diagnostics, cost, latency.                              |
| Ablations     | market-only, model-only, retrieval, bidirectional, Opus review, SAE, longshot guard.               |
| Results       | What beat market baseline, where retrieval helped/hurt, where models disagreed.                    |
| Discussion    | Forecasting as monitored probabilistic decision-making, not raw LLM prediction.                    |
| Failure cases | Stale sources, longshot narratives, near-resolution latency, parse failures.                       |

## 34. Open questions ledger

| Question                                      | Why it matters                                                 | Status                                  |
|-----------------------------------------------|----------------------------------------------------------------|-----------------------------------------|
| Are live updates/online learning allowed?     | Determines whether expert weights can adapt during evaluation. | Need organizer confirmation.            |
| Forecasting vs trading track final choice?    | Changes risk and output requirements.                          | Start compatible with both.             |
| Hosted endpoint required?                     | Determines whether FastAPI/VPS is needed.                      | Wait for final instructions.            |
| How much source context is allowed/available? | Affects retrieval and RAG design.                              | Build source gate anyway.               |
| Which domains dominate live set?              | Affects SAE area weights and routing.                          | Monitor once events appear.             |
| Can external market APIs be used?             | Affects Kalshi/Polymarket connector value.                     | Optional, read-only research initially. |
| Does Opus improve near-threshold decisions?   | Determines escalation budget.                                  | Offline ablation.                       |
| Does SAE shrinkage help buckets?              | Differentiated research angle.                                 | Build offline first.                    |

# 35. References and grounding notes

This document was built from uploaded project files and papers plus official documentation checks for current API concepts. The main internal files are baseline-strategy.md, budget-policy.md, operating pack, research-spine.md, and submission-checklist.md. The main research papers are LLM-as-a-Prophet / Prophet Arena and Makers and Takers: The Economics of the Kalshi Prediction Market.

| Reference                                                                  | Relevant ideas                                                                                                                                                             |
|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LLM-as-a-Prophet: Understanding Predictive Intelligence with Prophet Arena | Live benchmark, market baseline, Brier/ECE/return, source context, multi-horizon analysis, model conservatism, bidirectional elicitation, reasoning-to-probability rubric. |
| Makers and Takers: The Economics of the Kalshi Prediction Market           | Favorite-longshot bias, maker/taker asymmetry, quote-driven microstructure, low-price overestimation, market prices informative but biased.                                |
| Prophet Hacks baseline strategy                                            | Direct ai-prophet-core loop, conservative risk, edge threshold, trace logs.                                                                                                |
| Budget policy                                                              | Cost caps, frontier model routing, no open-ended loops, log every paid call.                                                                                               |
| Operating pack                                                             | Robust reproducible submission with one run command and dry loop before complexity.                                                                                        |
| Research spine                                                             | Scoring rules, calibration, retrieval credibility, online learning, prediction markets, observability.                                                                     |
| Submission checklist                                                       | Environment, run command, API lifecycle, risk, observability, packaging, post-event capture.                                                                               |

## 36. Compact agent context

---

The following condensed context can be pasted into coding-agent sessions.

Owner: Rob Sneiderman.  
Project: Prophet Hacks 2026 forecasting/trading agent.

Default system:  
- market prior  
- GPT-5.5 primary forecast  
- optional Claude Opus review  
- targeted evidence  
- logit stacking  
- SAE-inspired shrinkage  
- Kalshi longshot guard  
- strict risk gate  
- JSONL traces

Do not build a raw LLM oracle.  
Do not let the model decide trade size.  
Code owns edge, risk, sizing, and logging.

Use frontier models for final forecasts.  
Use cheaper models/rules only for routing, source labels,  
obvious skips, and ablations.

GitHub is source of truth.  
Every patch needs tests, command run, metric delta or  
observability gain, and no secret/package regression.

Monitoring is mandatory:  
trace completeness, Brier/ECE, market skill, price-bucket  
diagnostics, model disagreement, cost, parse failures.

Key modules:  
providers, prompts, source\_gate, source\_scoring, forecaster,  
bidirectional, ensemble, market\_blend, calibration,  
sae\_shrinkage, longshot\_guard, risk, monitor\_run,  
evaluate\_offline.

Immediate next commands:  
make verify  
make offline-eval  
run toy replay  
generate monitor summary



# 37. Immediate TODO checklist

| Priority | Task                                                             |
|----------|------------------------------------------------------------------|
| P0       | Add reports/status.md and make verify.                           |
| P0       | Implement trace schema validator.                                |
| P0       | Add market_blend.py and longshot_guard.py tests.                 |
| P0       | Add offline toy/pastcast evaluator with Brier/ECE/market skill.  |
| P1       | Add GPT-5.5 structured forecast adapter.                         |
| P1       | Add Opus review adapter with escalation gate.                    |
| P1       | Add source scoring and RAG separation.                           |
| P1       | Add price-bucket diagnostics.                                    |
| P2       | Add SAE shrinkage module.                                        |
| P2       | Add hyperparameter search.                                       |
| P2       | Add Slack webhook summary.                                       |
| P3       | Add external API/read-only connectors if signal value is proven. |

# Competition Landscape & Preparation Addendum

For the full Prophet Hacks Research and Execution Playbook - Prepared by Rob Sneiderman

## Purpose

This addendum clarifies who this event is probably targeting, what competitor methods to expect, and how that should shape Rob Sneiderman's preparation. The core read: this is not a generic app hackathon; it is a graduate-level AI forecasting / statistics / markets / agent-systems event.

## 1. Target audience and student/course pipeline

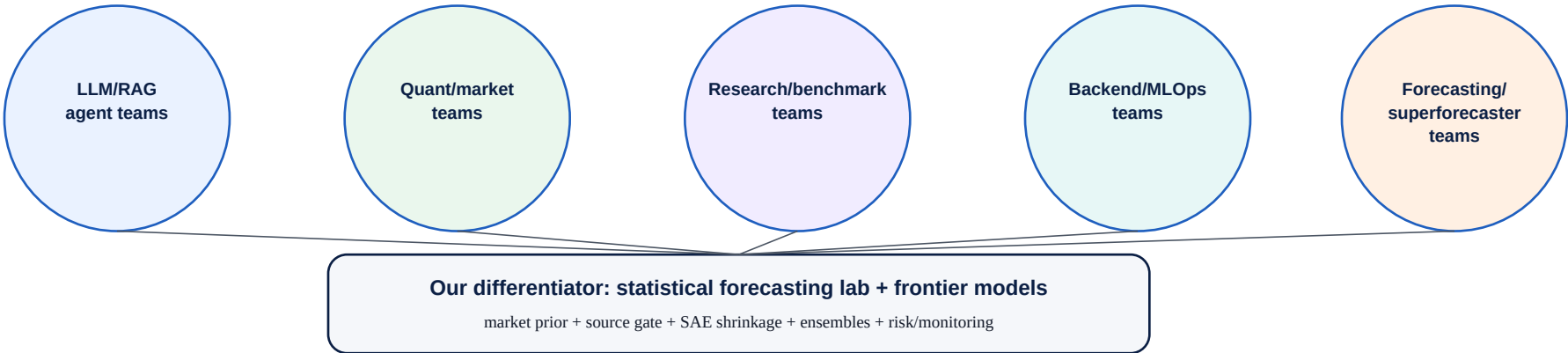
The event is built for a research-builder crowd. The official workshop frames forecasting as a rich ML tradition spanning time-series analysis, online learning, data-driven decisions, and quantitative finance, then asks whether general-purpose AI systems can reliably anticipate diverse real-world events. It also explicitly aims to bring together machine learning, statistics, economics, finance, and related communities.

| Pipeline / background            | What they bring                                              | What they may miss                                 | Our response                                                         |
|----------------------------------|--------------------------------------------------------------|----------------------------------------------------|----------------------------------------------------------------------|
| CS / ML / AI grad students       | LLM agents, RAG, model APIs, benchmark design                | calibration, market mechanics, risk discipline     | beat them with monitoring, market prior, and statistical calibration |
| Statistics / math / econometrics | proper scoring, uncertainty, shrinkage, SAE, causal thinking | agent orchestration, API packaging, live execution | turn Rob's statistics background into a system advantage             |
| Quant finance / economics        | market priors, EV, spreads, liquidity, microstructure        | LLM retrieval, source scoring, benchmark infra     | use Kalshi lessons plus frontier reasoning                           |
| Forecasting / superforecasting   | base rates, updates, scenario thinking, calibration habits   | automation at scale, CI, model routing             | encode their habits as prompts, logs, and calibration checks         |
| RL / online learning             | expert weighting, regret, adaptive policies                  | sparse delayed rewards, overengineering risk       | use RL-lite only: routing and expert weights                         |
| Backend / MLOps builders         | reliable loops, logs, CI, deployment                         | forecast probability quality                       | match their robustness while adding statistical edge                 |

## 2. What kind of competition this is

This is best treated as a live benchmark + agent engineering + probabilistic forecasting competition. The hackathon has forecasting and trading tracks evaluated through Prophet Arena, plus a Best PR to AI Prophet track. That means a serious submission can win in multiple ways: leaderboard performance, robust system design, and useful open-source contribution.

### Expected competitor archetypes and how we counter-position



| Competitor archetype                                                                                                                                                                                                                                                                             | Likely build                                            | Likely weakness                                      | How we exploit it                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|------------------------------------------------------|----------------------------------------------------------|
| LLM-agent teams                                                                                                                                                                                                                                                                                  | multi-agent RAG forecaster, debate prompts, web search  | source noise, weak calibration, hard-to-debug traces | simple monitored pipeline + source scoring + calibration |
| Quant/market teams                                                                                                                                                                                                                                                                               | market baseline, EV thresholds, spread-aware trades     | less strong on LLM evidence synthesis                | frontier model only when it can beat market prior        |
| Research teams                                                                                                                                                                                                                                                                                   | offline benchmark, plots, ablations, paper-style report | may not survive live API/tick loop                   | keep engineering path green first                        |
| Infra teams                                                                                                                                                                                                                                                                                      | CI, packaging, logs, dashboards                         | probability quality may be shallow                   | combine MLOps discipline with scoring rules/SAE          |
| Forecasting community teams                                                                                                                                                                                                                                                                      | base rates, scenario analysis, human-style updates      | not fully automated or source-gated                  | automate their strongest habits                          |
| <div><div>Default competitive stance</div><div>Do not try to be the flashiest agent. Try to be the most calibrated, monitored, source-aware, market-aware system. A boring-looking system that always knows why it acted can beat a dramatic system that cannot diagnose its errors.</div></div> |                                                         |                                                      |                                                          |

### 3. Course map -> repo modules

The workshop topics can be translated directly into modules. This keeps the work concrete and prevents paper-reading from becoming procrastination.

#### Skill stack: what the event implicitly tests

|                           |                                                        |
|---------------------------|--------------------------------------------------------|
| ML / foundation models    | model routing, prompts, tool use, reasoning effort     |
| Statistics / calibration  | Brier, ECE, reliability, SAE, shrinkage                |
| Markets / quant finance   | bid/ask, spreads, price buckets, longshot bias, EV     |
| Retrieval / NLP           | source ranking, evidence extraction, RAG, timestamping |
| Online learning / RL-lite | expert weighting, hard-case mining, regret framing     |
| MLOps / backend           | CI, logs, packaging, replay, dashboards                |

| Workshop/course area                                         | Module to build                                     | Metric/check                                       |
|--------------------------------------------------------------|-----------------------------------------------------|----------------------------------------------------|
| Architectures: agentic systems, LLM-as-a-Prophet             | forecasting/forecaster.py, providers.py, prompts.py | parse failure rate, forecast coverage              |
| Evaluation: metrics and benchmark design                     | evaluation/brier.py, ece.py, compare_variants.py    | Brier, ECE, skill vs market, CI                    |
| Reasoning: probabilistic, calibration, causal/temporal       | calibration.py, sae_shrinkage.py, horizon features  | reliability diagrams by domain/horizon             |
| Retrieval: search, credibility, RAG                          | source_gate.py, source_scoring.py                   | source quality lift, retrieved_at checks           |
| Foundations: scoring rules, online learning, decision theory | expert_pool.py, risk.py, edge.py                    | expert weights, EV/utility, regret-style summaries |
| Markets & society: prediction markets                        | longshot_guard.py, price_bucket_diagnostics.py      | price-bin Brier/return, longshot failures          |

## 4. What judges/organizers are likely to value

Because top teams can be invited to present at the ICML workshop, the project should read as both a working agent and a research artifact. The workshop call values architectures, evaluation, probabilistic reasoning, retrieval, foundations, prediction markets, and societal impact. Our report and repo should speak that language explicitly.

| They may value                                 | What to show in our artifact                                                   |
|------------------------------------------------|--------------------------------------------------------------------------------|
| Probabilistic forecasting, not binary guessing | Brier/ECE/market-return tables, reliability diagrams                           |
| Comparison to market baseline                  | market_only variant and skill-vs-market report                                 |
| Source-quality awareness                       | source records with credibility/staleness and lift analysis                    |
| Modular system design                          | separate retrieval, forecast, calibration, risk, monitoring modules            |
| Research insight                               | SAE-style borrowed strength, Kalshi longshot guard, hard-case mining           |
| Reproducibility                                | GitHub Actions, fixtures, raw hashes, clean package test                       |
| Open-source contribution                       | possible PR to AI Prophet: docs, validation, example bot, monitoring utilities |

Side-channel opportunity

The Best PR to AI Prophet track is not a distraction if scoped correctly. A good PR could be an example monitoring bot, schema validator, documentation fix, or trace-summary utility. It can support the main project and create a separate route to recognition.

# 5. Additional notes for prep

| Note                                              | Implication                                                                                                                                           |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Expect mixed skill levels, but serious top teams. | The median team may build a generic RAG agent. The top teams will understand market baselines, calibration, model routing, and benchmark constraints. |
| The event is research-adjacent.                   | Because it connects to an ICML workshop, a clear artifact matters. Keep the PDF, run summaries, and ablations clean enough to share.                  |
| Our comparative edge is unusual.                  | Rob has statistics/SAE/data engineering instincts plus AI-builder execution. Most teams will not combine these naturally.                             |
| Avoid false sophistication.                       | A multi-agent debate system is less impressive than a monitored forecast stack that beats market-only on holdout and explains every trade.            |
| Prepare a 4-page workshop-style summary early.    | Even if not submitted as a paper, it forces clarity: thesis, method, experiments, results, limitations.                                               |
| Track every failure as evidence.                  | A failed forecast with trace, source records, and calibration bucket is useful. A failed forecast with only a rationale is mostly noise.              |

|                              |                                                                                                                                                                                                                  |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Concrete next repo additions | reports/competition_landscape.md, reports/presentation_outline.md, tools/price_bucket_diagnostics.py, tools/source_lift_analysis.py, .github/workflows/offline-eval.yml, and a possible ai-prophet PR idea list. |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 6. Grounding references for this addendum

---

Official workshop/hackathon pages: the workshop is titled Forecasting as a New Frontier of Intelligence at ICML 2026, lists target communities across ML, statistics, economics, and finance, and lists topics that map directly to our modules. Prophet Hacks is a 30-hour Chicago/remote sprint with forecasting and trading tracks evaluated through Prophet Arena plus a Best PR to AI Prophet award.

Project papers/files: the Prophet Arena paper defines the event/context/prediction pipeline and metrics; the Kalshi paper motivates market microstructure and longshot diagnostics; Rob's baseline strategy, budget policy, operating pack, research spine, and submission checklist define the engineering constraints and risk posture.

Action: append this addendum to the full playbook and use it to orient coding agents before they touch repo architecture.